Rule Engine Operator Performance

🔷 1. Indexed Equality / Hash-Based (FASTEST 🚀)

Operator	Example	Indexing	Complexity	Notes
`==`	`status == "ACTIVE"`	✅ Hash index	O(1)	Best possible performance
`in`	`status in ("A","B")`	✅ Hash index	O(1)	Internally expands to equality checks
`containsKey` (maps)	`map containsKey "x"`	✅ Index	O(1)	Efficient for map lookups

✔️ **Best for:** fixed enums, IDs, status codes, country codes

🔷 2. Collection Membership (FAST with overhead)

Operator	Example	Indexing	Complexity	Notes
`memberOf`	`age memberOf $allowedAges`	⚠️ Partial	O(k)	Works but depends on dynamic list
`contains` (collection)	`$list contains x`	⚠️ Partial	O(k)	No hash index on RHS

✔️ **Best for:** small dynamic lists, rule parameters

🔷 3. Range / Comparison (INDEXED via Rete nodes)

x>a ∧ x<bx > a \;\land\; x < bx>a∧x<b

Operator	Example	Indexing	Complexity	Notes
`>` `<` `>=` `<=`	`age > 18`	✅ Beta index	O(log n) effective	Very efficient in Rete network
`between`	`age between 18 and 30`	✅ Beta index	O(log n)	Converted internally

✔️ **Best for:** numeric filtering, dates, thresholds

- Beta Index refers to an optimization technique used in the Beta (join) nodes of the Rete Network similar to Graph data structures (Component of Rule Engine used in FiringRule) to speed up rule matching
- In the RETE algorithm, the network is split into:
  - **Alpha network** → filters *single facts*
  - **Beta network** → joins *multiple facts together*

- So:
  - **Alpha = single-condition filtering**
  - **Beta = combining conditions across facts**

- A **Beta index** is an internal lookup structure (usually a hash-based or indexed map) used in the **Beta node** to avoid scanning all combinations when joining facts.

🔷 4. String Pattern Matching (NON-INDEXED)

string pattern match\text{string} \;\text{pattern match}

Operator	Example	Indexing	Complexity	Notes
`str[startsWith]`	`name str[startsWith] "Mr"`	❌	O(n)	Fast CPU op but scans all facts
`str[endsWith]`	`email str[endsWith] ".com"`	❌	O(n)	Slightly slower than equality
`str[equalsIgnoreCase]`	`name equalsIgnoreCase "john"`	⚠️ Partial	O(n)	Case normalization cost

✔️ **Best for:** low-volume filters, pre-filtered datasets

🔷 5. Regex Matching (SLOW)

Operator	Example	Indexing	Complexity	Notes
`matches`	`name matches \"Mr\\\\..*\"`	❌	O(n*m)	Full scan + regex engine

❌ Avoid in:

- high throughput rules
- large fact sets

🔷 6. Dynamic Expression / Inline Object Creation (WORST)

Pattern	Example	Indexing	Complexity	Notes
Inline collections	`new HashSet<>(Arrays.asList(...)).contains(x)`	❌	O(n)	Re-created every evaluation
Custom functions	`eval(...)` / scripted conditions	❌	O(n)	No rete optimization

❌ Worst because:

- no indexing
- object allocation every match
- prevents rule reuse optimization